

~minute sheet~

Inversion ~Yes.

Statics vs. Dynamics

We are going to split the life of a computer program into two phases:

- Static = anything that happens before running the program

(compile time)

e.g. parsing
optimizations
syntax checking
lexing
linking compiler?
type checking
static compilation
static analysis

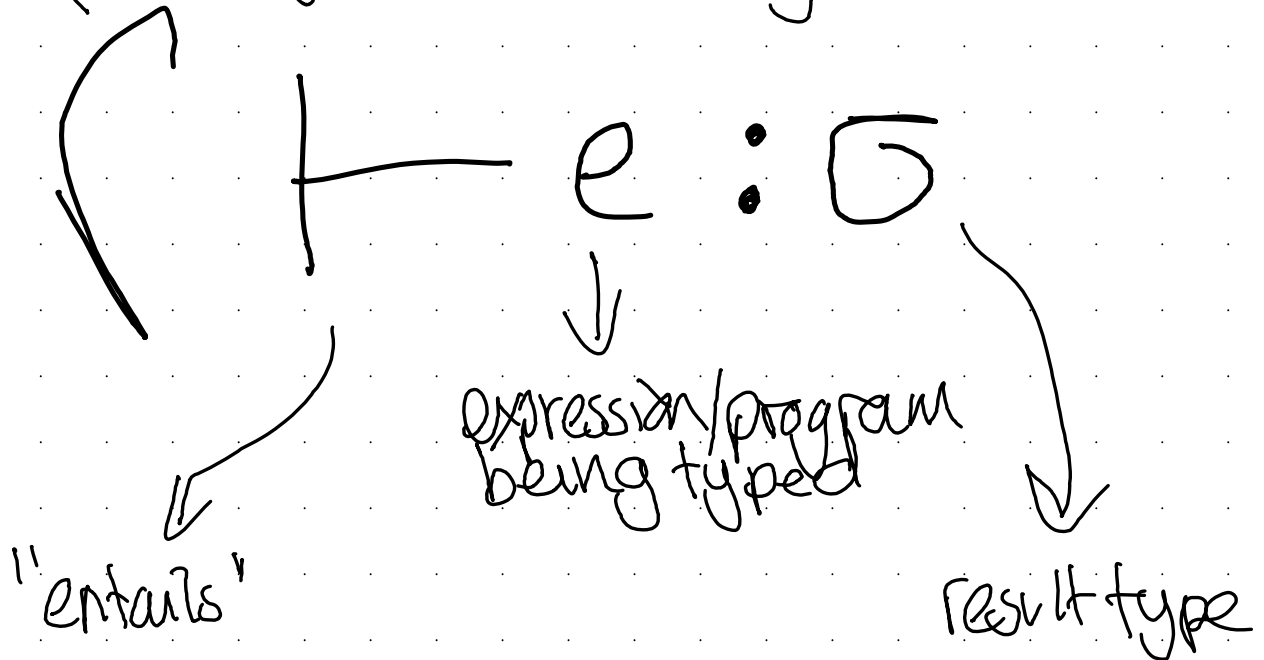
- dynamic = everything that happens when a program is running

(runtime)

e.g. calculating result
memory stuff
runtime error
side effects

Typing Judgements

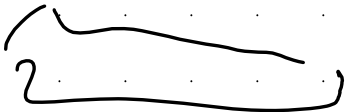
Context (environment):
set of variable type assignments
e.g. $\Gamma = \{x : T, y : G\}$



Terms = well-typed pre-terms

Binders

Binder = a language construct that closes over (binds) variables

eg. $\forall \exists \sum$ 

$\prod$  λ (var)

let
where

Bound variable = variable bound by a binder

Free variables = variables not bound by a binder

open: $\lambda x. xy$ — free
 binder bound

Closed expression = one with no free vars
Open expression = one with free variables

closed: $\lambda x. x$

α -equivalence/convertibility the name of a bound variable doesn't matter

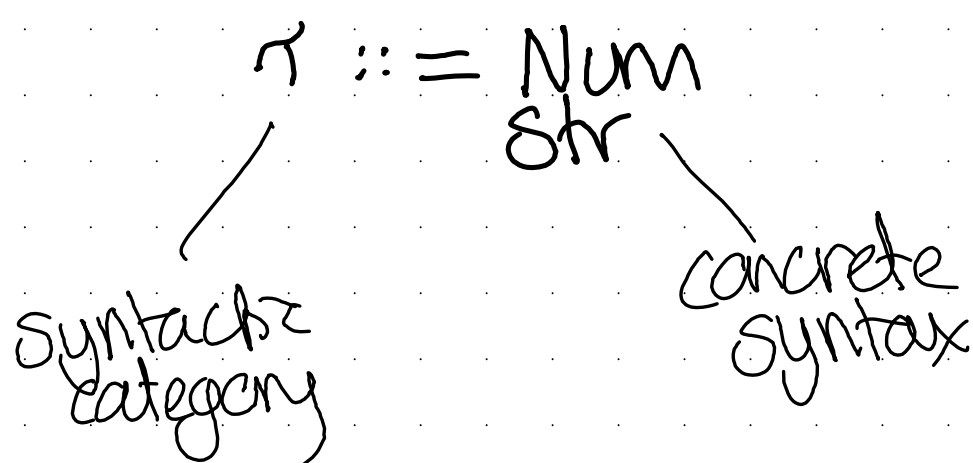
i.e. $\lambda x. x =_{\alpha} \lambda y. y$

$\lambda z. z =_{\alpha} \lambda a. a$

$\lambda xy. yx =_{\alpha} \lambda yx. ay$

A Little Language of Numbers + Strings - Syntax

Backus-Naur form (BNF)



Abstract syntax = the syntax of the term as it should appear in the abstract syntax tree (AST)

Concrete syntax = a user-friendly abbreviation of the abstract syntax

pre-term syntax category
 $e ::= x$

$\text{num}[n]$
 $\text{str}[s]$
 $\text{plus}(e_1; e_2)$
 $\text{mult}(e_1; e_2)$
 $\text{times}(e_1; e_2)$
 $\text{cat}(e_1; e_2)$
 $\text{len}(e)$
 $\text{let}(e_1; x.e_2)$

↑
abstract syntax

*

$e_1 + e_2$
 $e_1 \# e_2$
 $e_1 * e_2$
 $e_1 ++ e_2$
 $\text{len}(e)$
 $\text{let } x \leftarrow e_1 \text{ in } e_2$

↑
concrete syntax

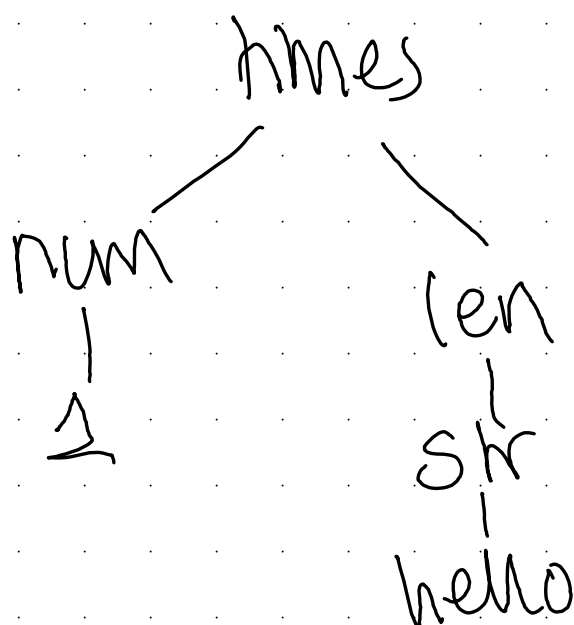
$n \in \mathbb{N}$ $s \in \Sigma^*$

e.g. $\Sigma = \{a, b, c, \dots, x\}$

e.g. $" " \in \Sigma^*$

$"hello" \in \Sigma^*$

$\text{times}(\text{num}[1]; \text{len}(\text{str}[\text{hello}]))$



Little lang - typing

$$\text{VAR} \frac{}{\Gamma, x:\sigma \vdash x:\sigma} \quad \exists \in$$

$$\text{NUM} \frac{n \in \mathbb{N}}{\Gamma \vdash \text{num}[n]:\text{Num}}$$

$$\text{STR} \frac{s \in \Sigma^*}{\Gamma \vdash \text{str}[s]:\text{Str}}$$

$$\text{PLUS} \frac{\Gamma \vdash e_1:\text{Num} \quad \Gamma \vdash e_2:\text{Num}}{\Gamma \vdash \text{plus}(e_1; e_2):\text{Num}}$$

$$\text{TIMES} \frac{\Gamma \vdash e_1:\text{Num} \quad \Gamma \vdash e_2:\text{Num}}{\Gamma \vdash \text{times}(e_1; e_2):\text{Num}}$$

$$\text{CAT} \frac{\Gamma \vdash e_1:\text{Str} \quad \Gamma \vdash e_2:\text{Str}}{\Gamma \vdash \text{cat}(e_1; e_2):\text{Str}}$$

$$\text{LEN} \frac{\Gamma \vdash e_1:\text{Str}}{\Gamma \vdash \text{len}(e_1):\text{Num}}$$

$$\text{LET} \frac{\Gamma \vdash e_1:\sigma_1 \quad \Gamma, x:\sigma_1 \vdash e_2:\sigma_2}{\Gamma \vdash \text{let}(e_1; x.e_2):\sigma_2}$$

Inversion

$$\text{PLUS} \frac{Pte_1:\text{Num} \quad Pte_2:\text{Num}}{P\text{plus}(e_1;e_2):\text{Num}}$$

inversion lemmata